VERSÃO 02.11.2026

STAGGY / ARQUITETURA DE SOFTWARE

DOCUMENTO DE ADRs

Registro de Decisões Arquiteturais
e histórico de revisões

001 — 014 | 21 registros de decisão, incluindo versões históricas

NAVEGAÇÃO DO DOCUMENTO

Índice das decisões

Decisões originais e revisões, preservadas na sequência de identificação.

001	Escolha do Prolog como mecanismo de cálculo para geração da grade de horários	3
002	Escolha do MySQL como banco de dados para o sistema	4
002.1	Escolha do MySQL como banco de dados para o sistema	5
003	Escolha do Express como framework para o back-end	6
004	Escolha do TypeScript como linguagem para o back-end	7
005	Escolha do React Native como framework para a aplicação mobile	8
005.1	Retirada da aplicação mobile do escopo atual e manutenção exclusiva da aplicação web	9
006	Escolha do Next.js como framework para a aplicação web	11
007	Comunicação síncrona entre as interfaces e a API nas operações de cadastro e consulta	12
007.1	Comunicação HTTP/JSON entre as interfaces e a API nas operações de cadastro, consulta e geração	14
008	Processamento assíncrono da geração de horários e consulta de andamento	16
008.1	Execução controlada da geração de horários sem fila persistente	18
009	Integração com o Prolog por processo externo e troca de dados em JSON	20
009.1	Integração com o Prolog por processo externo e troca de dados em JSON	22
010	Isolamento da integração com Prolog por meio do padrão Adapter	24
011	Adoção de arquitetura em camadas no back-end	26
011.1	Adoção de arquitetura em camadas no back-end	28
012	Recuperação de senha por token temporário de uso único	30
013	Escolha do Railway para hospedagem do banco de dados MySQL	32
013.1	Escolha do Azure para hospedagem do banco de dados MySQL	34
014	Autenticação por token de sessão persistido no banco de dados	36

REGISTRO DE DECISÃO ARQUITETURAL — ADR 001

Escolha do Prolog como mecanismo de cálculo para geração da grade de horários

NÚMERO	DATA	STATUS
001	10/09/2026	Aprovada

Contexto

A geração de grades de horários exige conciliar a disponibilidade dos professores, as disciplinas, as turmas, as salas e as preferências informadas. Quando realizada manualmente, essa atividade demanda conferências frequentes e ajustes para resolver conflitos.

O sistema precisa de um mecanismo capaz de analisar combinações de horários e encontrar uma grade que respeite as restrições obrigatórias, considerando as preferências sempre que possível.

Decisão e justificativa

Utilizar Prolog como mecanismo de cálculo da grade, integrado ao back-end da aplicação. A escolha considera os seguintes fatores:

- **Representação de regras:** A programação lógica permite expressar restrições como impedir que um professor ministre duas aulas simultaneamente ou seja alocado em um horário indisponível.
- **Busca por soluções:** O Prolog permite explorar combinações e buscar uma configuração que satisfaça as regras definidas. As preferências deverão ser traduzidas em critérios explícitos de seleção.
- **Controle da execução:** O cálculo poderá ocorrer na infraestrutura da aplicação, sem depender de serviços externos para gerar as grades.
- **Custos e exposição de dados:** A execução própria dispensa chamadas a APIs pagas de cálculo e reduz a necessidade de enviar informações dos professores a terceiros.

Alternativas consideradas

- **Elaboração manual:** Mantém a dependência de conferências e ajustes humanos, dificultando a redução do esforço operacional.
- **Soluções baseadas em IA:** Poderiam auxiliar na geração, mas a equipe priorizou um mecanismo com regras explícitas e maior controle sobre a validação dos resultados.

Consequências

A equipe precisará aprender Prolog, modelar as restrições e manter a integração com o back-end. Também será necessário tratar situações em que não exista uma grade válida para os dados informados.

O desempenho dependerá da quantidade de combinações e da implementação das regras. A redução do tempo de elaboração deverá ser verificada por testes, sem pressupor que a linguagem, por si só, garante processamento rápido.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 002

Escolha do MySQL como banco de dados para o sistema

NÚMERO	DATA	STATUS
002	15/09/2026	Revisada em 01/11/2026

Revisada em 01/11/2026 — substituída pela ADR 002.1 (originalmente aprovada)

Contexto

O sistema precisa armazenar professores, turmas, disciplinas, salas, disponibilidades e grades de horários. Essas informações possuem campos previamente definidos e relacionamentos que devem permanecer consistentes.

Além de consultar dados de diferentes entidades para alimentar o Prolog, será necessário salvar as grades geradas sem deixar registros incompletos em caso de falha.

Decisão e justificativa

Utilizar MySQL Community com o mecanismo de armazenamento InnoDB como banco de dados principal. A escolha considera os seguintes fatores:

- **Adequação ao modelo de dados:** A estrutura relacional permite representar os vínculos entre as entidades e consultar essas informações em conjunto.
- **Integridade dos registros:** Chaves primárias, estrangeiras e restrições ajudam a impedir referências inválidas e duplicidades. Por exemplo, uma aula não deverá estar vinculada a um professor inexistente.
- **Transações:** O InnoDB permite confirmar ou desfazer um conjunto de alterações, evitando que apenas parte de uma grade seja armazenada.
- **Facilidade de utilização:** SQL e ferramentas como MySQL Workbench apoiam a criação das tabelas, a execução de consultas e a inspeção dos dados.
- **Custo de adoção:** A edição Community permite utilizar o banco sem contratar uma edição comercial, favorecendo sua adoção no projeto acadêmico.

Alternativas consideradas

- **PostgreSQL:** Também atende aos requisitos relacionais e transacionais. O MySQL foi priorizado por atender ao projeto e pela facilidade de utilização considerada pela equipe.
- **MongoDB:** Sua flexibilidade documental não foi determinante, pois os dados possuem estrutura e relacionamentos previamente definidos.
- **SQLite:** É simples para aplicações locais e protótipos, mas menos favorável quando existem várias operações de escrita simultâneas.
- **Oracle Database:** Os requisitos atuais não justificam a complexidade de administração e os possíveis custos associados a uma edição comercial.

Consequências

Mudanças nos campos e relacionamentos exigirão alterações no esquema, migrações e possíveis ajustes na aplicação. A equipe também deverá manter backups, permissões de acesso e infraestrutura de hospedagem.

As restrições do banco contribuirão para a integridade dos dados, mas não substituirão as regras de escalonamento. Conflitos de horários deverão ser tratados pelo Prolog e pelas validações da aplicação.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 002.1

Escolha do MySQL como banco de dados para o sistema

NÚMERO	DATA	STATUS
002.1	01/11/2026	Aprovada

Aprovada — revisão da ADR 002

Contexto

O sistema precisa armazenar professores, turmas, disciplinas, salas, disponibilidades e grades de horários. Essas informações possuem campos previamente definidos e relacionamentos que devem permanecer consistentes.

Além de consultar dados de diferentes entidades para alimentar o Prolog, será necessário salvar as grades geradas sem deixar registros incompletos em caso de falha.

Decisão e justificativa

Utilizar MySQL com o mecanismo de armazenamento InnoDB como banco de dados relacional principal, hospedado no Azure Database for MySQL Flexible Server conforme a ADR 013.1. A escolha considera os seguintes fatores:

- **Adequação ao modelo de dados:** A estrutura relacional permite representar os vínculos entre as entidades e consultar essas informações em conjunto.
- **Integridade dos registros:** Chaves primárias, estrangeiras e restrições ajudam a impedir referências inválidas e duplicidades. Por exemplo, uma aula não deverá estar vinculada a um professor inexistente.
- **Transações:** O InnoDB permite confirmar ou desfazer um conjunto de alterações, evitando que apenas parte de uma grade seja armazenada.
- **Facilidade de utilização:** SQL e ferramentas como MySQL Workbench apoiam a criação das tabelas, a execução de consultas e a inspeção dos dados.
- **Compatibilidade com o serviço gerenciado:** A adoção do Azure mantém o modelo relacional e as consultas SQL já escolhidos, com custos associados ao serviço de hospedagem.

Alternativas consideradas

- **PostgreSQL:** Também atende aos requisitos relacionais e transacionais. O MySQL foi priorizado por atender ao projeto e pela facilidade de utilização considerada pela equipe.
- **MongoDB:** Sua flexibilidade documental não foi determinante, pois os dados possuem estrutura e relacionamentos previamente definidos.
- **SQLite:** É simples para aplicações locais e protótipos, mas menos favorável quando existem várias operações de escrita simultâneas.
- **Oracle Database:** Os requisitos atuais não justificam a complexidade de administração e os possíveis custos associados a uma edição comercial.

Consequências

Mudanças nos campos e relacionamentos exigirão alterações no esquema, migrações e possíveis ajustes na aplicação. A equipe também deverá manter migrações, backups, permissões e configurações de conexão com o serviço gerenciado do Azure. Os custos, a rede e a criptografia em trânsito deverão ser tratados conforme as ADRs 013.1 e 021.

As restrições do banco contribuirão para a integridade dos dados, mas não substituirão as regras de escalonamento. Conflitos de horários deverão ser tratados pelo Prolog e pelas validações da aplicação.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 003

Escolha do Express como framework para o back-end

NÚMERO	DATA	STATUS
003	15/09/2026	Proposta

Contexto

As aplicações web e mobile precisam de uma API para enviar informações, solicitar a geração de horários e consultar os resultados. Essa API deverá validar dados, controlar o acesso e coordenar a comunicação com o MySQL e o Prolog.

É necessário manter essas responsabilidades em um componente que possa evoluir independentemente das interfaces e atender futuras integrações.

Decisão e justificativa

Adotar Express, executado em Node.js e desenvolvido em TypeScript, para implementar a API do sistema. A escolha considera os seguintes fatores:

- **Tratamento de requisições:** O framework oferece recursos para definir rotas e organizar as operações disponibilizadas aos clientes.
- **Uso de middlewares:** Permite estruturar verificações de autenticação, autorização, validação e tratamento de erros.
- **Flexibilidade de organização:** As funcionalidades poderão ser separadas em módulos, como professores, disponibilidades e geração de horários.
- **Compatibilidade com a stack:** O uso de TypeScript permite manter a mesma linguagem no back-end e nas interfaces.
- **Integração entre componentes:** A API centralizará o acesso ao MySQL e o acionamento do Prolog, evitando que cada cliente implemente essas integrações.

Alternativas consideradas

- **Recursos de back-end do Next.js:** Permitiriam reunir interface e endpoints no mesmo projeto. Optou-se por uma API independente para atender web, mobile e futuras integrações.
- **NestJS:** Oferece maior padronização, mas exige aprendizado de convenções e abstrações adicionais, consideradas desnecessárias para o escopo inicial.
- **Node.js sem framework:** Permitiria construir o servidor diretamente, com mais implementação manual de rotas e tratamento das requisições.

Consequências

A equipe deverá definir padrões de organização e configurar os recursos de segurança, validação e acesso ao banco, pois o Express não estabelece uma arquitetura completa.

A separação entre API e interface exigirá configurações próprias de comunicação e implantação. A integração com o Prolog deverá prever execução assíncrona, tratamento de falhas e limites de tempo para que cálculos demorados não bloqueiem outras requisições.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 004

Escolha do TypeScript como linguagem para o back-end

NÚMERO	DATA	STATUS
004	15/09/2026	Proposta

Contexto

O back-end manipulará dados de professores, turmas, disciplinas e disponibilidades em diferentes operações. Alterações nessas estruturas podem introduzir incompatibilidades entre funções, serviços e integrações.

Busca-se uma linguagem compatível com Node.js e Express que aproveite conhecimentos de JavaScript e ajude a identificar parte desses problemas durante o desenvolvimento.

Decisão e justificativa

Desenvolver o back-end em TypeScript, definindo tipos e interfaces para os dados e contratos entre as camadas. O código será compilado para JavaScript. A escolha considera os seguintes fatores:

- **Proximidade com JavaScript:** O TypeScript preserva a sintaxe e os principais conceitos da linguagem, facilitando a adaptação da equipe.
- **Tipagem estática:** Permite identificar incompatibilidades antes da execução, como fornecer um texto a uma função que espera um identificador numérico.
- **Contratos explícitos:** Tipos e interfaces descrevem os dados esperados em cada operação, facilitando a compreensão do código.
- **Manutenção:** Alterações em estruturas e funções podem revelar, pela verificação de tipos, os pontos que precisam ser ajustados.
- **Padronização:** A linguagem também será utilizada nas interfaces, reduzindo a diversidade de tecnologias do projeto.

Alternativas consideradas

- **JavaScript:** Possui integração direta com Node.js e Express, mas não oferece a mesma verificação estática de tipos por padrão.
- **Java:** Oferece tipagem estática e recursos adequados para back-end, porém exigiria outro ecossistema e reduziria o aproveitamento do conhecimento em JavaScript.

Consequências

A equipe precisará aprender os recursos de tipagem, manter as definições de tipos e configurar a compilação. O uso excessivo de `any` deverá ser evitado, pois reduz os benefícios esperados.

A tipagem estática não valida automaticamente os dados recebidos durante a execução. Informações provenientes da API, de arquivos ou do Prolog continuarão exigindo validação antes de serem utilizadas.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 005

Escolha do React Native como framework para a aplicação mobile

NÚMERO	DATA	STATUS
005	30/09/2026	Revogada em 02/11/2026

Revogada em 02/11/2026 — substituída pela ADR 005.1 (originalmente aprovada)

Contexto

O projeto contará com uma aplicação mobile integrada à plataforma, com foco inicial em Android. A interface deverá consumir os serviços da API e permitir que os usuários acessem as funcionalidades disponibilizadas pelo sistema.

Busca-se aproveitar os conhecimentos de React e TypeScript utilizados no projeto, mantendo a possibilidade de desenvolver uma versão para iOS.

Decisão e justificativa

Utilizar React Native com TypeScript para desenvolver a aplicação mobile. A escolha considera os seguintes fatores:

- **Proximidade com React:** O uso de componentes, propriedades e estado permite aproveitar conceitos também utilizados no desenvolvimento web.
- **Compatibilidade com TypeScript:** Mantém a linguagem adotada nas demais camadas e permite definir os tipos dos dados utilizados pela interface.
- **Reutilização de componentes:** Elementos recorrentes da aplicação mobile poderão ser implementados uma vez e utilizados em diferentes telas.
- **Possibilidade de expansão:** Uma parte significativa do código poderá ser compartilhada entre Android e iOS, com adaptações conforme os recursos utilizados.
- **Integração com a API:** A aplicação consumirá os serviços do mesmo back-end utilizado pela interface web, mantendo as regras de negócio centralizadas.

Alternativas consideradas

- **Flutter:** Também permite desenvolvimento multiplataforma, mas exigiria aprender Dart e outro modelo de construção de interfaces.
- **Kotlin para Android nativo:** Atende ao Android, porém a interface desenvolvida dessa forma não seria reutilizada diretamente no iOS.

Consequências

A equipe deverá aprender os componentes e comportamentos próprios do ambiente mobile. A proximidade com React facilita o aprendizado, mas os componentes visuais da web não são reutilizados diretamente no React Native.

Uma eventual versão iOS exigirá configuração, testes e ajustes específicos. O compartilhamento de código reduz parte do trabalho, mas não elimina as diferenças entre plataformas.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 005.1

Retirada da aplicação mobile do escopo atual e manutenção exclusiva da aplicação web

NÚMERO	DATA	STATUS
005.1	02/11/2026	Aprovada

Aprovada — revoga a ADR 005

Contexto

A proposta anterior do Staggy contemplava aplicações web e mobile, com o uso de React Native e TypeScript para o desenvolvimento inicial em Android e possível expansão para iOS. Essa decisão foi registrada e aprovada na ADR 005.

Após a revisão dos objetivos do projeto, o escopo atual passou a concentrar-se na coleta de disponibilidades e preferências dos professores, na geração automática de grades de horários e nas funcionalidades acadêmicas relacionadas, por meio da aplicação web. O desenvolvimento de um aplicativo mobile deixou de fazer parte das entregas previstas.

É necessário formalizar a retirada dessa plataforma para evitar que o planejamento técnico, os requisitos e a documentação continuem considerando uma aplicação que pertence somente ao escopo anterior.

Decisão e justificativa

Retirar o desenvolvimento da aplicação mobile do escopo atual do Staggy e manter a **aplicação web, desenvolvida com Next.js, React e TypeScript, como interface prevista para os usuários**, conforme a ADR 006. A API em Express permanecerá responsável pelas regras e integrações do back-end.

A escolha considera os seguintes fatores:

- **Adequação ao escopo:** O aplicativo mobile fazia parte da proposta anterior e não é necessário para as entregas previstas na versão atual do projeto.
- **Concentração dos esforços:** A equipe poderá direcionar o desenvolvimento, os testes e a manutenção às funcionalidades web e ao mecanismo de geração de horários.
- **Redução de complexidade:** Dispensa a manutenção de um segundo projeto de interface, com configurações, testes e implantação próprios.
- **Consistência arquitetural:** Mantém a separação entre a aplicação web e a API, sem exigir a implementação de um cliente React Native.
- **Preservação do histórico:** A ADR 005 permanece documentada como uma decisão tomada para o escopo anterior, mas deixa de orientar o desenvolvimento atual.

Essa decisão não impede que a plataforma web seja acessada por navegadores em dispositivos móveis, quando houver suporte de interface, nem determina a criação futura de um aplicativo. Uma eventual retomada do mobile exigirá nova avaliação e decisão arquitetural.

Alternativas consideradas

- **Manter o desenvolvimento com React Native:** Preservaria a proposta anterior, mas exigiria esforço adicional com uma aplicação que não compõe o escopo atual.
- **Adiar o aplicativo mobile sem retirá-lo do escopo:** Manteria uma entrega indefinida no planejamento, dificultando a delimitação dos objetivos e responsabilidades da equipe.

- **Substituir React Native por outro framework mobile:** Alteraria a tecnologia, mas não resolveria a ausência de necessidade de uma aplicação mobile no escopo vigente.

Consequências

A ADR 005 passa ao status **Revogada** e permanece no documento exclusivamente como registro histórico da escolha anterior pelo React Native.

A equipe não precisará desenvolver, testar, hospedar ou publicar aplicativos Android ou iOS no escopo atual. As funcionalidades previstas deverão ser planejadas na aplicação web e na API existente.

Os documentos de requisitos, o planejamento e as ADRs que ainda mencionam aplicativos mobile deverão ser interpretados e, quando revisados, ajustados para refletir o escopo exclusivamente web. A ADR 006 continua válida.

A manutenção de uma API independente não dependerá da existência de um aplicativo mobile, pois continua oferecendo separação de responsabilidades e possibilidade de integrações futuras.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 006

Escolha do Next.js como framework para a aplicação web

NÚMERO	DATA	STATUS
006	30/09/2026	Aprovada

Contexto

A aplicação web precisa organizar telas para coleta de disponibilidades e preferências, solicitação da geração de horários e visualização das grades. Essas páginas compartilham elementos como cabeçalhos, menus e navegação.

Como React e TypeScript fazem parte da stack, busca-se uma estrutura que organize o desenvolvimento e facilite a manutenção das telas pela equipe.

Decisão e justificativa

Utilizar Next.js com React e TypeScript, adotando o App Router para organizar páginas e layouts. A interface consumirá a API em Express. A escolha considera os seguintes fatores:

- **Organização das rotas:** O roteamento baseado em arquivos oferece uma convenção para localizar e estruturar as páginas.
- **Layouts compartilhados:** Menus, cabeçalhos e outros elementos comuns poderão ser definidos em uma estrutura reutilizada entre telas.
- **Integração com React e TypeScript:** Permite manter as tecnologias escolhidas e aproveitar os conhecimentos da equipe.
- **Padronização do projeto:** As convenções do framework reduzem decisões repetitivas sobre configuração e organização, apoiando o trabalho colaborativo.

Alternativas consideradas

- **React com Vite e React Router:** Atenderia às necessidades com uma solução mais simples, mas exigiria definir separadamente as convenções de rotas e layouts.
- **Angular:** Oferece uma estrutura completa, porém exigiria substituir o ecossistema React já adotado.
- **Vue com Nuxt:** Também permite organizar páginas e componentes, mas demandaria aprender outro ecossistema de interface.

Consequências

A equipe deverá compreender o App Router e a separação entre componentes executados no servidor e no navegador. Recursos de renderização no servidor poderão aumentar a complexidade de implantação em comparação com uma interface totalmente estática.

As regras de negócio, o acesso ao MySQL e a integração com o Prolog permanecerão na API em Express. Essa divisão deverá ser mantida para evitar duplicação de responsabilidades.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 007

Comunicação síncrona entre as interfaces e a API nas operações de cadastro e consulta

NÚMERO	DATA	STATUS
007	12/10/2026	Revisada em 01/11/2026

Revisada em 01/11/2026 — substituída pela ADR 007.1

Contexto

As interfaces web e mobile precisam enviar disponibilidades e preferências, consultar dados necessários ao escalonamento e visualizar grades já geradas. Nessas operações, o usuário depende da resposta da API para confirmar uma ação ou apresentar informações.

Essas solicitações não envolvem a execução prolongada do cálculo em Prolog e podem ser atendidas no próprio ciclo de requisição e resposta.

Decisão e justificativa

Utilizar comunicação síncrona por HTTP/HTTPS, com dados em JSON, entre as interfaces e a API em Express nas operações de cadastro, atualização e consulta.

A interface enviará uma requisição e aguardará a resposta correspondente para confirmar o resultado da operação. Durante esse período, apresentará um indicador de carregamento, sem bloquear toda a navegação.

A escolha considera os seguintes fatores:

- **Confirmação da operação:** O usuário receberá uma indicação de sucesso ou erro após o processamento, como a confirmação de que suas disponibilidades foram salvas.
- **Simplicidade de implementação:** O fluxo de requisição e resposta atende às operações previstas sem exigir filas ou acompanhamento de tarefas.
- **Tratamento de validações:** A API poderá informar dados inválidos ou conflitos na mesma resposta, permitindo que o usuário faça as correções.
- **Contrato comum:** As aplicações web e mobile utilizarão os mesmos endpoints e formatos de dados.

A geração da grade continuará sendo processada de forma assíncrona. A consulta ao seu andamento será síncrona: cada requisição retornará o estado atual da tarefa.

Alternativas consideradas

- **Processamento assíncrono de todas as operações:** Exigiria registrar tarefas e acompanhar resultados até para cadastros e consultas simples, aumentando a complexidade sem necessidade identificada.
- **Comunicação persistente por WebSocket:** Permitiria troca contínua de mensagens, mas acrescentaria gerenciamento de conexões para operações atendidas pelo modelo de requisição e resposta.

Consequências

A interface dependerá da disponibilidade da API para concluir essas operações. Será necessário tratar carregamento, falhas de conexão e limites de tempo, além de evitar envios repetidos enquanto uma solicitação estiver em andamento.

Uma falha de conexão não significa necessariamente que a gravação foi desfeita. Por isso, operações de escrita deverão prever prevenção de duplicidades e confirmação do estado antes de uma nova tentativa, quando necessário.

A comunicação síncrona não exige bloquear a execução do Node.js ou a interface. Seu significado, nesta decisão, é que o resultado da operação será devolvido na resposta da própria requisição.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 007.1

Comunicação HTTP/JSON entre as interfaces e a API nas operações de cadastro, consulta e geração

NÚMERO	DATA	STATUS
007.1	01/11/2026	Proposta

Proposta — revisão da ADR 007

Contexto

As interfaces web e mobile precisam enviar disponibilidades e preferências, consultar dados necessários ao escalonamento e visualizar grades já geradas. Nessas operações, o usuário depende da resposta da API para confirmar uma ação ou apresentar informações.

Cadastros e consultas simples podem ser atendidos no próprio ciclo de requisição e resposta. A geração de horários também retornará sua conclusão na requisição correspondente, com limites de execução e tratamento apropriado para a sua duração variável.

Decisão e justificativa

Utilizar comunicação síncrona por HTTP/HTTPS, com dados em JSON, entre as interfaces e a API em Express nas operações de cadastro, atualização, consulta e solicitação de geração de horários.

A interface enviará uma requisição e aguardará a resposta correspondente para confirmar o resultado da operação. Durante esse período, apresentará um indicador de carregamento, sem bloquear toda a navegação.

A escolha considera os seguintes fatores:

- **Confirmação da operação:** O usuário receberá uma indicação de sucesso ou erro após o processamento, como a confirmação de que suas disponibilidades foram salvas.
- **Simplicidade de implementação:** O fluxo de requisição e resposta atende às operações previstas sem exigir filas ou acompanhamento de tarefas.
- **Tratamento de validações:** A API poderá informar dados inválidos ou conflitos na mesma resposta, permitindo que o usuário faça as correções.
- **Contrato comum:** As aplicações web e mobile utilizarão os mesmos endpoints e formatos de dados.

Na geração de horários, a requisição HTTP aguardará a conclusão ou interrupção do cálculo. A aplicação iniciará o Prolog em um processo externo e o acompanhará de modo não bloqueante, com timeout configurável, conforme as ADRs 008.1 e 009.1. Não haverá fila persistente nem polling de tarefas.

Alternativas consideradas

- **Processamento assíncrono de todas as operações:** Exigiria registrar tarefas e acompanhar resultados até para cadastros e consultas simples, aumentando a complexidade sem necessidade identificada.
- **Comunicação persistente por WebSocket:** Permitiria troca contínua de mensagens, mas acrescentaria gerenciamento de conexões para operações atendidas pelo modelo de requisição e resposta.

Consequências

A interface dependerá da disponibilidade da API para concluir essas operações. Será necessário tratar carregamento, falhas de conexão e limites de tempo, além de evitar envios repetidos enquanto uma solicitação estiver em andamento.

Uma falha de conexão não significa necessariamente que a gravação foi desfeita. Por isso, operações de escrita deverão prever prevenção de duplicidades e confirmação do estado antes de uma nova tentativa, quando necessário.

A comunicação síncrona não exige bloquear a execução do Node.js ou a interface. Seu significado, nesta decisão, é que o resultado da operação será devolvido na resposta da própria requisição.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 008

Processamento assíncrono da geração de horários e consulta de andamento

NÚMERO	DATA	STATUS
008	12/10/2026	Rejeitada em 01/11/2026

Rejeitada em 01/11/2026 — substituída pela ADR 008.1

Contexto

O tempo necessário para gerar uma grade pode variar conforme a quantidade de professores, turmas, salas e restrições. Manter uma requisição HTTP aberta durante todo o cálculo pode causar expiração da conexão e dificultar o acompanhamento pelo usuário.

Por outro lado, operações como salvar disponibilidades e consultar grades já concluídas precisam de uma resposta imediata sobre seu resultado.

Decisão e justificativa

Utilizar comunicação síncrona nas operações de cadastro e consulta e processamento assíncrono para a geração de horários.

Ao receber uma solicitação de geração, a API registrará a tarefa no MySQL e retornará seu identificador com o código HTTP `202 Accepted`, indicando que o processamento foi aceito, mas ainda não foi concluído.

Um worker separado da API buscará as tarefas pendentes e executará o cálculo por meio da integração com Prolog. A interface consultará periodicamente o andamento, utilizando *polling*, até que a tarefa alcance um estado final.

A escolha considera os seguintes fatores:

- **Tempo de cálculo variável:** A geração poderá continuar sem depender da duração de uma conexão HTTP.
- **Disponibilidade da API:** A execução do cálculo em um processo separado permite que a API continue atendendo outras operações, respeitados os recursos disponíveis.
- **Acompanhamento pelo usuário:** O identificador da tarefa permite consultar o andamento mesmo após atualizar a página.
- **Persistência das solicitações:** O registro no MySQL mantém as tarefas identificáveis após reinicializações e permite implementar recuperação de processamentos interrompidos.
- **Simplicidade inicial:** O uso do banco já adotado e de consultas periódicas evita acrescentar um serviço de mensageria ou uma conexão permanente com a interface.

Alternativas consideradas

- **Geração síncrona:** Simplificaria a implementação inicial, mas manteria a requisição aberta durante o cálculo e aumentaria a exposição a limites de tempo.
- **WebSocket ou Server-Sent Events:** Permitiriam enviar atualizações à interface sem consultas periódicas, porém exigiriam manter conexões e tratar reconexões. Para o acompanhamento inicial, o polling é suficiente.
- **Fila em um serviço de mensageria:** Ofereceria recursos próprios para distribuição e processamento de tarefas, mas adicionaria infraestrutura que não se justifica no escopo inicial.

Consequências

Será necessário controlar os estados das tarefas, como pendente, processando, concluída, sem solução, falha e tempo esgotado. A interface deverá apresentar esses estados de forma clara e interromper as consultas quando o processamento terminar.

O worker deverá reservar cada tarefa de forma atômica, limitar execuções simultâneas e permitir identificar tarefas interrompidas. A recuperação após falhas precisará evitar a gravação duplicada de resultados.

O polling produzirá consultas adicionais à API e um pequeno intervalo entre a conclusão do cálculo e sua apresentação ao usuário. A frequência dessas consultas deverá equilibrar a atualização da interface e a carga no servidor.

Essa abordagem exige manter um processo de worker além da API. Caso o volume de gerações aumente, a fila no MySQL e a consulta periódica poderão ser reavaliadas.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 008.1

Execução controlada da geração de horários sem fila persistente

NÚMERO	DATA	STATUS
008.1	01/11/2026	Proposta

Proposta — substitui a ADR 008

Contexto

A geração automática de horários depende do processamento de restrições relacionadas a professores, disciplinas, turmas, salas, disponibilidades e preferências. O tempo necessário para encontrar uma solução pode variar conforme a complexidade dos dados.

A estratégia anteriormente proposta previa um worker independente, uma fila persistente no MySQL e consultas periódicas de andamento. Entretanto, o projeto foi simplificado e não utilizará uma tabela específica para gerenciar tarefas de geração.

É necessário controlar a execução do cálculo, impedir processamentos indefinidos e garantir que falhas não resultem em grades incompletas.

Decisão e justificativa

Executar a geração de horários a partir de uma requisição à API Express, que coordenará o cálculo realizado pelo SWI-Prolog por meio de um processo externo.

A requisição permanecerá aberta até a conclusão ou interrupção do processamento. O Node.js utilizará operações assíncronas para acompanhar o processo externo, sem bloquear sua thread principal enquanto aguarda o resultado.

A escolha considera os seguintes fatores:

- **Simplicidade arquitetural:** Elimina a necessidade de uma fila persistente, polling e worker específico para gerenciar tarefas.
- **Controle de tempo:** Cada geração terá um limite de execução configurável, definido e ajustado por testes de desempenho.
- **Tratamento de resultados:** A API distinguirá uma grade encontrada, ausência comprovada de solução, tempo esgotado e falha técnica.
- **Controle de concorrência:** O sistema deverá limitar execuções simultâneas e impedir gerações conflitantes para o mesmo contexto acadêmico.
- **Persistência transacional:** A grade somente será salva após a validação do resultado, utilizando transações no MySQL.

O tempo de execução deverá respeitar também os limites de conexão da infraestrutura HTTP. Em caso de timeout, o processo Prolog deverá ser encerrado e seus recursos liberados.

Alternativas consideradas

- **Worker com fila persistida no MySQL:** Permitiria acompanhamento e recuperação de tarefas, mas exigiria estados adicionais, controle de workers e persistência das solicitações. Essa complexidade não foi considerada necessária para o escopo atual.

- **Serviço de mensageria:** Ofereceria distribuição de tarefas e maior escalabilidade, porém acrescentaria componentes de infraestrutura sem demanda identificada.
- **Execução do cálculo diretamente no processo Node.js:** Evitaria criar processos externos, mas não corresponde à integração definida com SWI-Prolog e poderia comprometer o isolamento do cálculo.

Consequências

A resposta da geração dependerá da conclusão do cálculo dentro do limite estabelecido. Requisições poderão falhar caso a infraestrutura encerre a conexão antes do processamento.

Será necessário configurar timeout, encerramento do processo, limite de concorrência e tratamento padronizado das diferentes situações de término.

A interrupção por timeout não deverá ser interpretada como inexistência de solução. Da mesma forma, uma falha durante a persistência não poderá deixar uma grade parcialmente gravada.

A execução não será automaticamente recuperada após uma interrupção do servidor. Uma nova tentativa deverá ser solicitada e controlada pela aplicação.

Caso a complexidade ou o volume de gerações aumente, o processamento por fila poderá ser reavaliado.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 009

Integração com o Prolog por processo externo e troca de dados em JSON

NÚMERO	DATA	STATUS
009	12/10/2026	Revisada em 01/11/2026

Revisada em 01/11/2026 — substituída pela ADR 009.1

Contexto

O back-end precisa encaminhar ao Prolog os dados de professores, turmas, disciplinas, salas e disponibilidades, além de receber a grade calculada. Essa integração deve permitir acompanhar a execução e identificar falhas sem bloquear o atendimento das demais requisições.

Como o tempo de cálculo pode variar conforme as restrições e a quantidade de combinações, a execução será integrada ao processamento assíncrono das solicitações de geração.

Decisão e justificativa

Executar o SWI-Prolog como um processo externo, iniciado por um worker desenvolvido em Node.js e TypeScript. O worker enviará os dados em JSON pela entrada padrão do processo e receberá o resultado, também em JSON, pela saída padrão.

O Express ficará responsável por registrar as solicitações e disponibilizar seu andamento. O worker coordenará a execução do Prolog e a persistência dos resultados no MySQL.

A escolha considera os seguintes fatores:

- **Simplicidade de integração:** O Node.js permite iniciar processos externos por meio de `spawn`, dispensando a criação de uma API HTTP exclusiva para o Prolog.
- **Contrato de dados definido:** O JSON permite estabelecer uma estrutura explícita para os dados de entrada e os resultados, com recursos de leitura e escrita disponíveis no SWI-Prolog.
- **Separação de responsabilidades:** O Prolog concentra as regras de escalonamento, enquanto o worker controla a execução e o armazenamento. O acesso ao banco permanece na aplicação Node.js.
- **Execução em segundo plano:** A solicitação HTTP pode ser respondida antes da conclusão do cálculo, permitindo que o usuário acompanhe o andamento posteriormente.
- **Isolamento entre gerações:** Cada tarefa poderá iniciar seu próprio processo Prolog, evitando o compartilhamento involuntário de estado entre cálculos.

Alternativas consideradas

- **Prolog como serviço HTTP:** Permitiria executar o motor em uma infraestrutura separada, mas exigiria manter outro serviço, sua comunicação de rede e seus controles de acesso. Essa complexidade não se justifica no escopo inicial.
- **Troca de dados por arquivos:** Poderia utilizar arquivos JSON de entrada e saída, mas exigiria gerenciar nomes, permissões e limpeza dos arquivos de cada geração.
- **Execução durante a requisição HTTP:** Simplificaria o fluxo inicial, porém manteria a conexão aberta durante o cálculo e aumentaria a possibilidade de atingir limites de tempo.

Consequências

O ambiente do worker deverá ter o SWI-Prolog instalado e permitir a criação de processos. A integração será concentrada em um componente próprio, facilitando futuras mudanças na forma de executar o motor.

Será necessário definir limites de tempo, quantidade de execuções simultâneas e mecanismos para impedir que a mesma solicitação seja processada por dois workers. Mensagens de diagnóstico deverão utilizar a saída de erro, mantendo a saída padrão reservada ao JSON de resposta.

O worker deverá validar o resultado recebido e distinguir geração concluída, ausência de solução, falha técnica e tempo esgotado. Caso o prazo termine antes da conclusão da busca, o sistema não deverá afirmar que uma grade válida é impossível.

A execução de um processo por tarefa adiciona custo de inicialização e consumo de recursos. Se o volume de gerações crescer ou surgir a necessidade de hospedar o Prolog separadamente, essa decisão poderá ser reavaliada.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 009.1

Integração com o Prolog por processo externo e troca de dados em JSON

NÚMERO	DATA	STATUS
009.1	01/11/2026	Proposta

Proposta — revisão da ADR 009

Contexto

O back-end precisa encaminhar ao Prolog os dados necessários para gerar uma grade e receber o resultado calculado.

A integração deve permitir controlar o processo, validar os dados trocados e identificar falhas sem bloquear a execução de outras operações da API.

Considerando a substituição da arquitetura com worker e fila persistente, a comunicação com o Prolog deverá ser coordenada diretamente pelo serviço de geração.

Decisão e justificativa

Executar o SWI-Prolog como processo externo iniciado pela aplicação Node.js, utilizando `spawn`.

Os dados serão enviados em JSON pela entrada padrão do processo, e o resultado será recebido também em JSON pela saída padrão. A saída de erro ficará reservada a diagnósticos.

A escolha considera os seguintes fatores:

- **Integração direta:** Dispensa a criação de um serviço HTTP independente para o Prolog.
- **Contrato explícito:** O JSON permite definir e validar os formatos de entrada e saída.
- **Isolamento de execução:** O processamento ocorre em um processo separado da API.
- **Controle operacional:** O Node.js poderá acompanhar, limitar e encerrar a execução.
- **Separação de responsabilidades:** O Prolog realizará o cálculo, enquanto o back-end coordenará a execução e a persistência.

Alternativas consideradas

- **Prolog como serviço HTTP independente:** Permitiria executar o motor em outra infraestrutura, mas exigiria administrar mais um serviço e sua comunicação pela rede.
- **Comunicação por arquivos temporários:** Permitiria armazenar dados intermediários, porém exigiria controlar a criação, os nomes, as permissões e a exclusão dos arquivos.
- **Integração direta com bibliotecas nativas:** Poderia evitar processos externos, mas criaria dependências adicionais entre os ambientes de execução.

Consequências

A infraestrutura responsável pela geração deverá possuir o SWI-Prolog instalado e permitir a execução de processos externos.

Será necessário validar o JSON recebido, controlar a quantidade de dados enviados, tratar encerramentos anormais e garantir a liberação dos recursos utilizados.

O resultado do Prolog deverá ser validado antes de ser persistido. A comunicação continuará encapsulada pelo Adapter definido na ADR-010.

A substituição do mecanismo de cálculo continuará possível, desde que outro componente implemente o contrato esperado pela aplicação.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 010

Isolamento da integração com Prolog por meio do padrão Adapter

NÚMERO	DATA	STATUS
010	12/10/2026	Proposta

Contexto

A geração de horários exige converter os dados da aplicação para o formato utilizado pelo Prolog, executar o cálculo e interpretar a resposta. Se esses detalhes ficarem distribuídos pelos serviços do back-end, mudanças na comunicação com o motor poderão exigir alterações em várias partes do sistema.

É necessário definir uma interface de integração que permita ao serviço de geração solicitar o cálculo sem depender dos detalhes de execução do SWI-Prolog.

Decisão e justificativa

Utilizar o padrão Adapter para encapsular a comunicação com o Prolog. A aplicação utilizará um contrato com uma operação de geração de grade, recebendo dados estruturados e retornando um resultado padronizado.

O adapter será responsável por converter os dados de entrada, acionar o processo externo, interpretar a resposta e traduzir falhas técnicas. A coordenação da tarefa e a persistência no MySQL permanecerão nos serviços da aplicação, enquanto as regras de escalonamento continuarão no Prolog.

A escolha considera os seguintes fatores:

- **Redução do acoplamento:** O serviço de geração dependerá do contrato de integração, sem conhecer comandos, caminhos de arquivos ou canais de entrada e saída do processo.
- **Centralização das conversões:** A transformação entre os dados da aplicação e os formatos do Prolog ficará em um único componente.
- **Facilidade de testes:** O adapter poderá ser substituído por uma implementação simulada nos testes do serviço, permitindo verificar diferentes resultados sem executar o motor de cálculo.
- **Manutenção da integração:** Alterações na forma de comunicação, como uma futura migração para HTTP, poderão ficar concentradas no adapter, desde que o contrato seja preservado.

Alternativas consideradas

- **Integração diretamente no serviço de geração:** Exigiria menos componentes inicialmente, mas misturaria a coordenação do caso de uso com detalhes de execução e conversão de dados.
- **Funções auxiliares sem contrato definido:** Poderiam centralizar parte do código, porém ofereceriam menos clareza sobre as operações e os resultados esperados pela aplicação. Essas funções também poderiam implementar o Adapter se passassem a cumprir um contrato explícito.

Consequências

Será necessário manter o contrato de integração, os formatos de dados e a implementação do adapter. Essa separação adiciona uma camada ao código, mas limita a propagação de mudanças relacionadas ao Prolog.

Os testes do serviço poderão utilizar respostas simuladas, enquanto testes de integração deverão verificar a comunicação real com o motor. O adapter deverá distinguir uma grade encontrada, ausência de solução, tempo esgotado e falha técnica, preservando essas diferenças para a aplicação.

A substituição do mecanismo de cálculo não será automática: um novo motor precisará atender ao contrato e às regras esperadas pelo sistema.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 011

Adoção de arquitetura em camadas no back-end

NÚMERO	DATA	STATUS
011	14/10/2026	Revisada em 01/11/2026

Revisada em 01/11/2026 — substituída pela ADR 011.1

Contexto

O back-end do sistema precisa receber disponibilidades e preferências, registrar solicitações de geração, coordenar o cálculo em Prolog e armazenar as grades no MySQL.

Essas responsabilidades serão utilizadas tanto pela API em Express quanto pelo worker responsável pelo processamento assíncrono. Sem uma separação definida, consultas ao banco, tratamento de requisições e integração com Prolog podem ficar misturados, dificultando a manutenção e os testes.

Decisão e justificativa

Organizar o back-end em camadas, agrupando o código por funcionalidade e separando as seguintes responsabilidades:

Camada	Responsabilidade
Entrada	Controllers recebem requisições HTTP e devolvem respostas. O worker recebe tarefas pendentes e inicia seu processamento.
Aplicação	Serviços coordenam os casos de uso, como registrar uma geração, processar uma tarefa e consultar uma grade.
Infraestrutura	Componentes de acesso a dados realizam consultas e gravações no MySQL. O adapter concentra a comunicação com Prolog.

Controllers e worker utilizarão os serviços para executar os casos de uso. Os serviços coordenarão o acesso aos dados e ao adapter, sem incorporar consultas SQL ou detalhes de execução do Prolog.

As regras de combinação e restrição dos horários permanecerão no Prolog. Os serviços em TypeScript cuidarão das regras do fluxo, como verificar condições para iniciar uma geração e atualizar seu estado.

A escolha considera os seguintes fatores:

- **Separação de responsabilidades:** Cada parte do código terá uma função definida, evitando concentrar todo o processamento nos controllers ou no worker.
- **Reutilização:** A coordenação dos casos de uso poderá ser utilizada por diferentes pontos de entrada, sem duplicar regras.
- **Facilidade de manutenção:** Mudanças nas consultas ou na comunicação com Prolog ficarão concentradas nos componentes correspondentes.
- **Testabilidade:** Será possível testar os serviços com substitutos controlados para o acesso ao banco e para o motor de cálculo.
- **Adequação ao escopo:** A divisão oferece organização suficiente para a geração de horários sem exigir uma estrutura extensa de abstrações.

Alternativas consideradas

- **Organização sem separação em camadas:** Permitiria iniciar com menos arquivos, mas favoreceria a mistura de responsabilidades e dificultaria alterações conforme o projeto evoluísse.
- **MVC como estrutura principal:** É uma alternativa válida, porém exige detalhar onde ficam a coordenação das tarefas, a persistência e a integração com Prolog. A divisão escolhida torna essas responsabilidades explícitas.
- **Arquitetura Hexagonal ou Clean Architecture:** Oferecem maior isolamento por contratos e regras de dependência, mas sua adoção completa acrescentaria abstrações consideradas desnecessárias para o escopo inicial.

Consequências

A equipe deverá manter os limites entre as camadas: controllers não executarão consultas SQL, componentes de persistência não tratarão respostas HTTP e o adapter não coordenará o fluxo completo da geração.

A separação aumentará a quantidade de arquivos e funções. Para evitar complexidade desnecessária, não será obrigatório criar classes, interfaces ou serviços que apenas repassem chamadas sem uma finalidade clara.

As camadas serão divisões lógicas do código, sem exigir serviços independentes para cada uma. API e worker poderão compartilhar os mesmos módulos, embora sejam executados como processos separados.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 011.1

Adoção de arquitetura em camadas no back-end

NÚMERO	DATA	STATUS
011.1	01/11/2026	Proposta

Proposta — revisão da ADR 011

Contexto

O back-end precisa realizar autenticação, gerenciar professores, formulários, turmas, solicitações, grades e operações relacionadas à geração automática de horários.

O código atual apresenta controllers que realizam consultas diretamente ao banco de dados. Embora essa abordagem permita implementar funcionalidades rapidamente, pode dificultar a reutilização das regras de negócio, os testes e a manutenção conforme o sistema cresce.

É necessário estabelecer uma separação clara entre tratamento de requisições, execução das regras da aplicação e integração com recursos externos.

Decisão e justificativa

Organizar o back-end em camadas lógicas, agrupando o código por funcionalidade e separando as seguintes responsabilidades:

Camada	Responsabilidade
Entrada	Controllers recebem requisições, validam os contratos HTTP e devolvem respostas
Aplicação	Serviços coordenam casos de uso, validações e operações transacionais
Infraestrutura	Repositories acessam o MySQL e adapters integram componentes externos, como o Prolog

Os controllers deverão delegar a execução das operações aos serviços, evitando incorporar regras de negócio complexas ou consultas SQL diretamente.

Os serviços coordenarão as operações e utilizarão os componentes de infraestrutura necessários.

O acesso ao MySQL será concentrado em repositories, enquanto o acesso ao Prolog continuará isolado pelo Adapter.

A escolha considera os seguintes fatores:

- **Separação de responsabilidades:** Reduz a mistura entre HTTP, regras de aplicação, persistência e integração externa.
- **Testabilidade:** Permite testar os serviços com dependências simuladas.
- **Manutenção:** Alterações na persistência e na integração com Prolog ficam mais localizadas.
- **Reutilização:** Os casos de uso poderão ser executados sem depender diretamente dos controllers.
- **Adequação ao projeto:** Não exige uma adoção completa de arquiteturas mais complexas.

Alternativas consideradas

- **Controllers com acesso direto ao banco:** Exigem menos componentes inicialmente, mas favorecem a concentração de responsabilidades e dificultam os testes isolados.
- **MVC tradicional:** É adequado para aplicações web, porém não define sozinho a organização dos casos de uso e das integrações externas.
- **Clean Architecture ou Arquitetura Hexagonal completas:** Oferecem maior isolamento, mas poderiam introduzir abstrações adicionais consideradas desnecessárias para o estágio atual.

Consequências

Será necessário reorganizar gradualmente os módulos existentes, evitando interromper o desenvolvimento das funcionalidades.

Controllers não deverão concentrar consultas SQL, repositories não deverão produzir respostas HTTP e adapters não deverão coordenar casos de uso completos.

A separação será lógica, sem exigir a criação de serviços independentes para cada camada.

Não será obrigatório utilizar classes ou interfaces para todas as operações. Funções e contratos simples poderão ser empregados quando oferecerem uma solução mais adequada.

A adoção do padrão Repository será detalhada em uma ADR específica.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 012

Recuperação de senha por token temporário de uso único

NÚMERO	DATA	STATUS
012	18/10/2026	Proposta

Contexto

Os usuários precisam recuperar o acesso quando esquecerem a senha. O processo deve comprovar o acesso ao e-mail cadastrado, limitar a validade da autorização e impedir a reutilização do link.

Também é necessário evitar que o formulário de recuperação revele quais e-mails possuem conta no sistema.

Decisão e justificativa

Utilizar links HTTPS de recuperação contendo um token aleatório, gerado de forma criptograficamente segura, com validade inicial de **30 minutos** e uso único.

O banco armazenará somente o hash do token, sua finalidade, o usuário associado, a expiração e o estado de utilização. O token de recuperação será independente dos tokens de sessão e de convite.

O fluxo seguirá estas etapas:

1. O usuário informa o e-mail e recebe uma resposta genérica, independentemente da existência da conta.
2. Para uma conta elegível, o sistema envia o link ao endereço cadastrado.
3. O usuário abre a página de recuperação e informa uma nova senha.
4. A API valida o token e, na confirmação, atualiza o hash da senha e consome o token de forma atômica.
5. O sistema invalida os demais tokens de recuperação, revoga as sessões e notifica a alteração por e-mail.
6. O usuário realiza um novo login.

A escolha considera os seguintes fatores:

- **Autorização limitada:** O token permite apenas redefinir a senha durante o prazo estabelecido.
- **Prevenção de reutilização:** Após a confirmação, o mesmo link não poderá alterar novamente a senha.
- **Proteção das contas cadastradas:** Respostas genéricas e tratamento com duração semelhante reduzem a exposição da existência de usuários.
- **Compatibilidade com web e mobile:** A mesma página web de recuperação poderá ser aberta por ambos, simplificando a implementação inicial.

Alternativas consideradas

- **Código temporário por e-mail:** É viável, mas exige digitação e controles específicos de tentativas; o link oferece um fluxo mais direto.
- **Perguntas de segurança:** Dependem de informações que podem ser conhecidas ou descobertas por terceiros.
- **Envio de senha provisória:** Acrescenta o gerenciamento de uma credencial temporária e a obrigação de substituí-la após o acesso.

Consequências

Será necessário configurar o envio de e-mails, limitar solicitações e tentativas de validação e impedir o registro dos tokens em logs. Os links serão construídos com o endereço confiável da aplicação previamente configurado.

A simples abertura do link não consumirá o token; o consumo ocorrerá somente na confirmação da nova senha. A recuperação não autenticará automaticamente o usuário nem reativará contas desativadas.

O processo dependerá do acesso ao e-mail cadastrado. A recuperação quando esse acesso também tiver sido perdido deverá seguir um procedimento separado, ainda a ser definido.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 013

Escolha do Railway para hospedagem do banco de dados MySQL

NÚMERO	DATA	STATUS
013	18/10/2026	Rejeitada em 01/11/2026

Rejeitada em 01/11/2026 — substituída pela ADR 013.1

Contexto

O sistema precisa de um banco de dados disponível remotamente para atender à API e ao worker de geração de horários, sem depender do computador de um integrante da equipe.

Considerando o escopo acadêmico e o prazo de desenvolvimento, busca-se uma hospedagem que simplifique a implantação e permita concentrar o esforço nas funcionalidades do sistema, mantendo controle sobre persistência, acesso e recuperação dos dados.

Decisão e justificativa

Utilizar o Railway para hospedar o MySQL, com armazenamento persistente e configuração de backups. A plataforma disponibiliza um template para provisionar o banco e recursos de backup de volumes. Railway Docs

O banco será acessado pela API e pelo worker, sem conexão direta das interfaces web e mobile. As credenciais serão mantidas na configuração dos ambientes, fora do código-fonte.

A escolha considera os seguintes fatores:

- **Facilidade de implantação:** O template de MySQL reduz o trabalho inicial de instalação e configuração do banco.
- **Disponibilidade remota:** Permite utilizar uma instância centralizada, independentemente dos computadores dos desenvolvedores.
- **Compatibilidade com a stack:** Mantém o MySQL já escolhido, sem exigir alteração do modelo relacional ou da linguagem de consulta.
- **Adequação ao projeto:** A simplicidade de provisionamento favorece a entrega inicial, considerando o tempo disponível e a necessidade de limitar o esforço com infraestrutura.
- **Persistência:** O uso de volume persistente permite conservar os dados entre recriações do serviço, sem substituir a necessidade de backups. Railway Docs

A decisão prioriza simplicidade operacional. Não pressupõe que o Railway seja gratuito ou mais barato que todas as alternativas; os custos deverão ser avaliados conforme o plano e os recursos utilizados.

Alternativas consideradas

- **Microsoft Azure — Azure Database for MySQL Flexible Server:** Oferece um serviço gerenciado de MySQL. É uma alternativa adequada, mas a equipe priorizou o fluxo de implantação do Railway para o escopo inicial. A adoção do Azure exigiria avaliar suas configurações de rede, capacidade e administração. Microsoft Learn
- **Amazon AWS — Amazon RDS for MySQL:** Disponibiliza MySQL gerenciado e recursos de backup e recuperação. Também atende ao projeto, porém demandaria configurar e administrar os recursos correspondentes no ambiente AWS. Amazon Relational Database Service
- **Servidor virtual próprio:** Ofereceria maior controle sobre o ambiente, mas transferiria à equipe a instalação, atualização, proteção e recuperação do banco.

- **Hospedagem no computador de um integrante:** Reduziria o custo inicial de infraestrutura, mas criaria dependência da disponibilidade, da conexão e da configuração desse computador.

Consequências

O funcionamento do sistema dependerá da disponibilidade da plataforma e da conectividade com o banco. A equipe deverá acompanhar consumo, armazenamento e limites do plano contratado.

Será necessário configurar backups e verificar sua restauração. O armazenamento persistente protege contra a perda decorrente de recriações comuns do serviço, mas não evita exclusões acidentais ou alterações incorretas nos dados.

Quando API e worker estiverem no mesmo ambiente Railway, será priorizada a comunicação pela rede privada. Acessos externos necessários deverão utilizar configuração segura e credenciais com permissões limitadas.

A decisão será reavaliada se os custos, a latência ou as necessidades de disponibilidade e operação deixarem de atender ao projeto. Uma eventual migração exigirá planejar a transferência dos dados, a compatibilidade da versão do MySQL e a atualização das conexões.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 013.1

Escolha do Azure para hospedagem do banco de dados MySQL

NÚMERO	DATA	STATUS
013.1	01/11/2026	Aprovada

Aprovada — substitui a ADR 013 (Railway)

Contexto

O Staggy precisa de um banco de dados disponível remotamente para armazenar os cadastros acadêmicos, as disponibilidades, as respostas aos formulários e as grades de horários.

Inicialmente, o Railway havia sido considerado como alternativa de hospedagem pela simplicidade de implantação. Entretanto, em 21/10/2026, a plataforma divulgou um incidente de segurança envolvendo possível acesso não autorizado a variáveis de ambiente, incluindo informações que poderiam conter credenciais e chaves de acesso.

O comunicado da plataforma recomendou a substituição das credenciais potencialmente expostas. A decisão de migrar para o Azure foi tomada pela equipe após reavaliar o risco associado ao fornecedor.

Diante desse cenário, a equipe decidiu substituir a escolha anterior e priorizar um serviço gerenciado de banco de dados no Microsoft Azure.

Decisão e justificativa

Utilizar o Azure Database for MySQL Flexible Server para hospedar o banco de dados principal do Staggy.

A aplicação continuará utilizando o MySQL e o mecanismo de armazenamento InnoDB, preservando a modelagem relacional escolhida anteriormente.

O acesso ao banco ocorrerá exclusivamente pelos componentes autorizados do back-end, sem conexões diretas das interfaces web e mobile.

A escolha considera os seguintes fatores:

- **Segurança:** O serviço oferece conexões criptografadas por TLS, controles de acesso e opções de restrição de conectividade.
- **Serviço gerenciado:** Reduz a responsabilidade da equipe sobre instalação e manutenção do mecanismo de banco.
- **Compatibilidade:** Mantém o MySQL já adotado sem exigir substituição do modelo de dados.
- **Backups e recuperação:** Disponibiliza backups automáticos e restauração para pontos no tempo dentro do período configurado.
- **Controle operacional:** Permite configurar capacidade, armazenamento, rede e monitoramento conforme as necessidades do sistema.

O Azure oferece criptografia das conexões por TLS e backups automáticos, mas a configuração de permissões, rede, credenciais e capacidade continuará sob responsabilidade da equipe.

Alternativas consideradas

- **Railway:** Inicialmente priorizado pela simplicidade de implantação. Foi descartado após a reavaliação de segurança motivada pelo incidente divulgado em outubro de 2026.

- **Amazon RDS for MySQL:** Também oferece hospedagem gerenciada, mecanismos de backup e controles de segurança. Permanece tecnicamente adequado, mas não foi selecionado pela equipe.
- **Servidor virtual próprio:** Ofereceria maior controle sobre a infraestrutura, porém exigiria administrar diretamente a instalação, as atualizações, os backups e a proteção do banco.
- **Hospedagem local:** Reduziria a necessidade inicial de contratar infraestrutura, mas criaria dependência de computadores e conexões particulares dos integrantes.

Consequências

A equipe deverá configurar a instância no Azure, controlar o acesso pela rede e manter as credenciais fora do código-fonte.

A aplicação Node.js deverá estabelecer conexões seguras com o banco, validando os certificados TLS de forma adequada.

Será necessário monitorar custos, utilização de recursos e limites de armazenamento, já que a contratação do serviço não implica gratuidade.

Os backups automáticos deverão ser configurados conforme a necessidade do projeto, e a capacidade de restauração deverá ser testada.

Os ambientes de desenvolvimento e de produção deverão utilizar credenciais e configurações separadas.

Caso credenciais reais do Staggy tenham sido armazenadas no Railway, elas deverão ser substituídas e seus acessos revisados.

A decisão não pressupõe que o Azure seja imune a incidentes de segurança. A proteção dos dados continuará dependendo da configuração correta da infraestrutura e da aplicação.

REGISTRO DE DECISÃO ARQUITETURAL — ADR 014

Autenticação por token de sessão persistido no banco de dados

NÚMERO	DATA	STATUS
014	01/11/2026	Proposta

Contexto

O Staggy possui usuários com diferentes permissões, incluindo administradores e professores. O sistema precisa autenticar essas pessoas e garantir que apenas usuários autorizados acessem as funcionalidades correspondentes.

Também é necessário controlar a duração das sessões, permitir logout e impedir que contas desativadas continuem utilizando o sistema.

O back-end já utiliza tokens aleatórios de sessão, armazenando seus hashes no MySQL e verificando a validade a cada requisição autenticada.

Decisão e justificativa

Adotar autenticação baseada em tokens aleatórios de sessão, gerados de forma criptograficamente segura e validados pelo servidor.

Cada sessão será registrada no MySQL, contendo o usuário associado, o hash do token, a data de criação, a expiração e as informações necessárias para controlar sua revogação.

A escolha considera os seguintes fatores:

- **Revogação imediata:** Permite invalidar uma sessão diretamente pelo banco.
- **Controle centralizado:** A API verifica se a sessão permanece válida e se o usuário está ativo.
- **Armazenamento protegido:** O banco armazena o hash do token, e não seu valor original.
- **Expiração por inatividade:** A validade poderá ser renovada quando houver atividade autenticada.
- **Compatibilidade:** A autenticação poderá ser compartilhada pelas aplicações web e mobile.

Será mantido inicialmente o prazo de uma hora de inatividade já utilizado pelo back-end. Também deverá ser definido um limite máximo absoluto para a duração das sessões.

O mecanismo atual de transmissão por `Authorization: Bearer` poderá continuar sendo utilizado. Entretanto, a estratégia de armazenamento do token na aplicação web precisará ser formalizada, evitando sua exposição indevida. Cookies `HttpOnly`, `Secure` e `SameSite`, com proteção contra CSRF quando aplicável, são uma alternativa a avaliar para o navegador; no mobile, o armazenamento deve utilizar mecanismos seguros do sistema operacional.

Alternativas consideradas

- **JWT sem persistência de sessão:** Permitiria validar tokens assinados sem consultar uma sessão no banco, mas dificultaria a revogação imediata individual sem mecanismos adicionais.
- **Sessões exclusivamente em memória:** Simplificariam o armazenamento inicial, mas seriam perdidas após reinicializações e dificultariam a execução em múltiplas instâncias.
- **Serviço externo de identidade:** Poderia centralizar a autenticação e oferecer recursos adicionais, porém criaria outra dependência e custos potenciais para o projeto.

Consequências

Cada requisição autenticada dependerá da validação da sessão e das permissões do usuário.

Será necessário manter mecanismos de logout, expiração, revogação e limpeza de sessões antigas.

A alteração de senha deverá invalidar as sessões conforme a ADR-012, e a desativação de uma conta deverá impedir seu uso imediatamente.

A equipe deverá proteger os tokens contra exposição em logs, URLs e armazenamento inseguro no navegador.

A validação de sessão não substituirá a autorização de acesso aos dados, que deverá considerar também os vínculos institucionais.